

WEEK-5

dirent.h is the header in the C POSIX library for the C programming language that contains constructs that facilitate directory traversing.

int **closedir**(DIR* dirp)

Closes the directory stream referred to by dirp. Upon return, dirp may no longer point to an accessible object of the type DIR. If a file descriptor is used to implement type DIR, that file descriptor will be closed. Upon successful completion, closedir() returns 0. Otherwise, -1 is returned and errno is set to indicate the error

DIR* **opendir**(const char* dirname)

Opens a directory stream corresponding to the directory named by dirname. The directory stream is positioned at the first entry. If the type DIR is implemented using a file descriptor, applications will only be able to open up to a total of OPEN_MAX files and directories. Upon successful completion, opendir() returns a pointer to an object of type DIR. Otherwise, a null pointer is returned and errno is set to indicate the error.

struct dirent* **readdir**(DIR* dirp)

Returns a pointer to a structure representing the directory entry at the current position in the directory stream specified by the argument dirp, and positions the directory stream at the next entry. It returns a null pointer upon reaching the end of the directory stream. If entries for dot or dot-dot exist, one entry will be returned for dot and one entry will be returned for dot-dot; otherwise they will not be returned. When an error is encountered, a null pointer is returned and errno is set to indicate the error. When the end of the directory is encountered, a null pointer is returned and errno is not changed.

The memory location pointed to by the return value is managed by the library and may change on subsequent calls to readdir. It should not be free'd by the user.

1. Write program using the I/O system calls of UNIX/LINUX operating system (opendir, readdir).

Aim: Program Purpose is to use I/O System calls of Unix Operating system (opendir, readdir)

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <dirent.h>
```

```
int main() {
```

```
    DIR *dir;
```

```
    struct dirent *entry;
```

```
    // Open the current directory
```

```

dir = opendir(".");
if (dir == NULL) {
    perror("opendir");
    exit(EXIT_FAILURE);
}
// Read directory entries
printf("Contents of current directory:\n");
while ((entry = readdir(dir)) != NULL) {
    printf("%s\n", entry->d_name);
}
// Close the directory
if (closedir(dir) == -1) {
    perror("closedir");
    exit(EXIT_FAILURE);
}
return 0;
}

```

OUTPUT:

mgit@DESKTOP-6FUTF8Q:~/OS\$./a.out

Contents of current directory:

system.c

a.out

..

.

2. Write a C program to copy the contents of one file to another file using System calls(Open,write,read,close)

Aim:

```
#include<unistd.h>
```

```
#include<syscall.h>
```

```
#include<stdio.h>
```

```
#include<fcntl.h>

void main()

{int fd1,fd2,n;

char src[13],dest[13],buff[50];

printf("enter the source\n");

scanf("%s",src);

printf("enter the dest\n");

scanf("%s",dest);

fd1=open(src,O_RDONLY);

if(fd1<0)

{

printf("Source is not existiung file hence cannot copy the contents\n");

}

fd2=open(dest,O_WRONLY|O_CREAT);

n=read(fd1,buff,sizeof(buff));

buff[n]='\0';

write(fd2,buff,n);

printf("contents of %s file are copied to %s file",src,dest);

close(fd1);

close(fd2);

}
```

OUTPUT:

mgit@DESKTOP-6FUTF8Q:~\$./a.out

enter the source

abc.c

enter the dest

ab.c

contents of abc.c file are copied to ab.c file

3. Write a C program to use stat() syscall

Aim: C program to find file permission, size, creation and last modification date of a given file.

```
#include <stdio.h>

#include <unistd.h>

#include <sys/stat.h>

#include <time.h>

void printFileProperties(struct stat stats);

int main()
{
    char path[100];

    struct stat stats;

    printf("Enter source file path: ");

    scanf("%s", path);


    // stat() returns 0 on successful operation,
    // otherwise returns -1 if unable to get file properties.
    if (stat(path, &stats) == 0)
    {
        printFileProperties(stats);
    }
    else
    {
        printf("Unable to get file properties.\n");
        printf("Please check whether '%s' file exists.\n", path);
    }

    return 0;
}


/* Function to print file properties. */
void printFileProperties(struct stat stats)
```

```
{ struct tm dt;

// File permissions
printf("\nFile access: ");
if (stats.st_mode & R_OK)
    printf("read ");
if (stats.st_mode & W_OK)
    printf("write ");
if (stats.st_mode & X_OK)
    printf("execute");

// File size
printf("\nFile size: %d", stats.st_size);

// Get file creation time in seconds and
// convert seconds to date and time format
dt = *(gmtime(&stats.st_ctime));
printf("\nCreated on: %d-%d-%d %d:%d:%d", dt.tm_mday, dt.tm_mon, dt.tm_year + 1900,
        dt.tm_hour, dt.tm_min, dt.tm_sec);

// File modification time
dt = *(gmtime(&stats.st_mtime));
printf("\nModified on: %d-%d-%d %d:%d:%d", dt.tm_mday, dt.tm_mon, dt.tm_year + 1900,
        dt.tm_hour, dt.tm_min, dt.tm_sec);

}
```